

When Blockchain and Federated Learning Hit it Off: Forging a Novel Proof of Useful Work

Hengrui Lu , Kai Fan , *Member, IEEE*, Zhanpeng Guo , Kuan Zhang , *Member, IEEE*,
and Hui Li , *Member, IEEE*

Abstract—Traditional Proof-of-Work (PoW) consensus secures blockchains by incurring significant energy and resource wastage on solving valueless puzzles. The Proof-of-Useful-Work (PoUW) paradigm emerges to repurpose this effort for tasks of societal or scientific value. However, existing PoUW approaches face a critical trade-off between the work’s practical utility and its verifiability, as efficiently validating complex computations without full re-execution remains a major open problem. This paper proposes Proof-of-Useful-Federated-Work (PoUFW), a novel consensus protocol that resolves this dilemma by recasting the “work” as Federated Learning (FL) model aggregation. Specifically, PoUFW mandates that miners not only perform the aggregation, but also generate a cryptographic proof of its correctness. To achieve this, we propose a novel proof mechanism that builds upon the Kate-Zaverucha-Goldberg (KZG) polynomial commitment scheme to generate a concise, non-interactive proof. This mechanism enables validators to rapidly verify the aggregation’s integrity without re-executing the computation or accessing the underlying model. Consequently, PoUFW transforms the computational overhead of consensus into valuable, verifiable machine learning outcomes, while simultaneously establishing a decentralized, incentivized framework for FL. Rigorous theoretical analysis substantiates the protocol’s security and correctness. Extensive simulations further demonstrate that PoUFW achieves high verification efficiency alongside robust security, confirming its practical feasibility and scalability. This work provides a robust theoretical and empirical foundation for building blockchain systems with improved resource efficiency and societal value.

Index Terms—Blockchain, consensus algorithm, federated learning, verifiable computation, proof-of-useful-work.

Received 4 August 2025; revised 14 April 2026; accepted 17 June 2026. Date of publication 25 June 2026; date of current version 1 September 2026. This work was supported in part by the 111 Center (B16037), in part by the National Key R&D Program of China under Grant 2024YFB3108101, in part by the National Natural Science Foundation of China under Grant 62372356, in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2023ZD018, in part by the Key Research and Development Program of Shaanxi under Grant 2021ZDLGY06-02, in part by the Natural Science Foundation of Shaanxi Province under Grant 2019ZDLGY1202, in part by Shaanxi Innovation Team Project under Grant 2018TD-007, in part by Xi’an Science and Technology Innovation Plan under Grant 20189168CX9JC10, and in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02038. (*Corresponding author: Kai Fan.*)

Hengrui Lu, Kai Fan, Zhanpeng Guo, and Hui Li are with the State Key Laboratory of Integrated Services Networks, School of Cyber Engineering, Xidian University, Xi’an 710071, China (e-mail: hrlu@stu.xidian.edu.cn; kfan@mail.xidian.edu.cn; zhanp.guo@gmail.com; lihui@mail.xidian.edu.cn).

Kuan Zhang is with the Department of Electrical and Computer Engineering, University of Nebraska-Lincoln, Lincoln, NE 68588 USA (e-mail: kuan.zhang@unl.edu).

Digital Object Identifier 10.1109/TDSC.2026.3707157

I. INTRODUCTION

BLOCKCHAIN technology relies on consensus algorithms to establish trustless agreement and secure decentralized ledgers [1], [2]. Proof-of-Work (PoW) pioneered this paradigm by requiring miners to competitively solve resource-intensive cryptographic puzzles [3]. However, PoW harbors a fundamental contradiction: its robust security demands vast, continuous computational power, yet the work itself—billions of arbitrary hash calculations—yields no intrinsic socio-economic utility beyond network maintenance. The sheer scale of this “resource burning,” exemplified by the Bitcoin network’s estimated 167.68 TWh annual electricity consumption [4], demands a paradigm shift toward value-added consensus mechanisms.

To address this computational idling, Proof-of-Useful-Work (PoUW) [5] replaces arbitrary hashing with tasks generating tangible societal outputs. Yet, realizing practical PoUW introduces a critical design dilemma: balancing the task’s *usefulness* with efficient, decentralized *verifiability* [6]. Generating reliable, lightweight proofs for complex computations without forcing validators into computationally prohibitive full re-executions remains a formidable open challenge.

Federated Learning (FL) aggregation presents a compelling candidate to elegantly resolve this tension [7]. Standard FL preserves privacy by enabling clients to train models locally, while a central server aggregates the updates [8]. This centralized architecture, however, inevitably introduces Single Point of Failure (SPOF) vulnerabilities [9] and lacks robust open-network participation incentives [10]. Recasting the computationally intensive FL aggregation as the “work” in PoUW creates a symbiotic integration as conceptually illustrated in Fig. 1: it transforms wasteful cryptographic hashing into valuable machine learning computation, while supplanting the vulnerable central aggregator with a competitive, decentralized network of miners economically incentivized by block rewards [11], [12], [13].

Despite this conceptual appeal, securely integrating FL into a robust consensus protocol is highly non-trivial. Pioneering frameworks like PoFL [14], PF-PoFL [15], and TORR [16] predominantly rely on game-theoretic incentives rather than rigorous cryptographic guarantees. Substituting PoW’s deterministic security with fragile economic assumptions of participant rationality leaves networks vulnerable to complex collusion. Crucially, these methods lack a direct, lightweight mechanism to verify aggregation correctness; validators must either blindly trust the miner or perform costly re-computations. Bridging

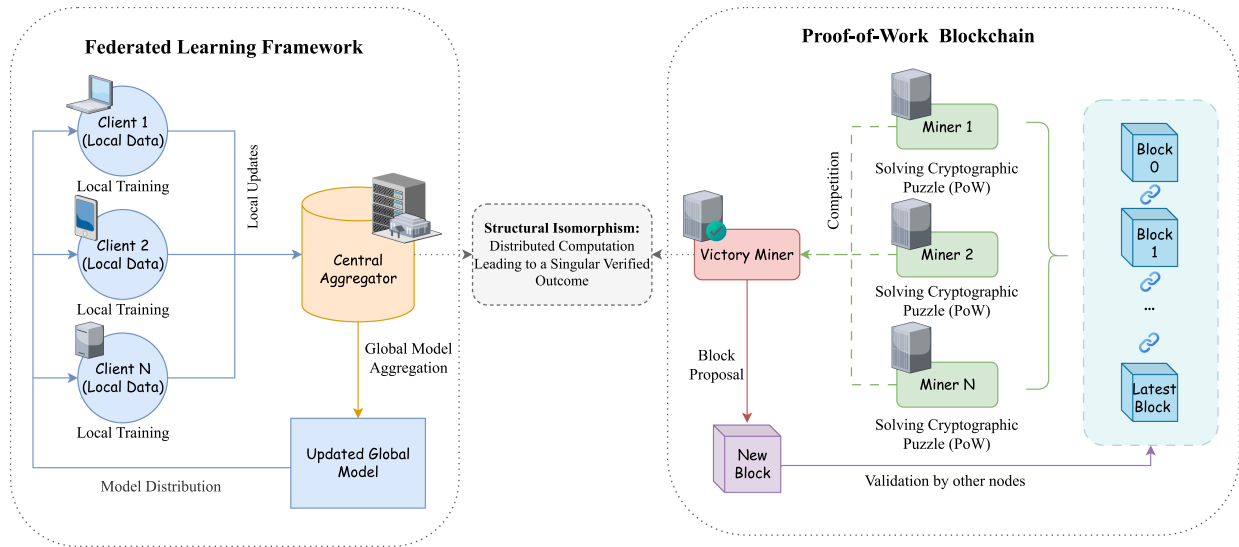


Fig. 1. Conceptual Comparison Highlighting Structural Parallels between Federated Learning and PoW Blockchain.

this gap requires inextricably embedding PoW’s core security properties—efficient verification and tamper-proof state binding—directly into the useful task via deterministic cryptographic proofs.

To address these challenges, this paper proposes Proof-of-Useful-Federated-Work (PoUFW), a novel consensus protocol that anchors blockchain security directly to verifiable FL aggregation. By integrating the Kate-Zaverucha-Goldberg (KZG) polynomial commitment scheme [17], PoUFW empowers miners to generate concise, non-interactive cryptographic proofs of their complex aggregation computations. This mechanism enables validators to confirm structural integrity without re-execution or raw data access, flawlessly merging consensus security with valuable computational outcomes.

The main contributions of this paper are:

- We propose PoUFW, substituting standard PoW puzzles with FL aggregation. This pioneers a shift from game-theoretic FL-blockchain systems to a rigorous cryptographic framework, linking block proposal rights directly to the provably correct execution of the aggregation task.
- We propose a novel verification mechanism that renders the complex FL aggregation task efficiently verifiable. Our method systematically maps client model updates into polynomials and leverages the homomorphic properties of the KZG scheme, enabling a miner to generate a single, constant-size, non-interactive proof that attests to the correctness of the entire aggregation, thereby decoupling verification cost from the complexity of the useful work.
- We design a Contextual Proof Binding (CPB) mechanism utilizing the Fiat-Shamir heuristic to derive the cryptographic challenge directly from the candidate block’s unique context. This mathematically anchors each proof to a specific chain state, thwarting replay attacks and inheriting PoW’s intrinsic tamper-resistance.
- Comprehensive simulations demonstrate that PoUFW achieves highly asymmetric computational complexity.

Verification remains exceptionally lightweight even as the FL task difficulty scales, confirming the protocol’s practical feasibility and scalability for real-world deployments.

The remainder of this paper is organized as follows. Section II reviews the related literature on Proof-of-Useful-Work and its integration with Federated Learning. Section III outlines the core cryptographic primitives that underpin our protocol. In Section IV, we detail the system model, security assumptions, and overarching design objectives. The core design of the PoUFW protocol is presented in Section V, specifying the roles and interactions of all participants. We provide a rigorous security analysis and evaluate the protocol’s computational complexity in Section VI. Section VII presents an extensive experimental evaluation to validate the practical performance and scalability of our approach. Finally, Section VIII concludes the paper with a summary of our contributions.

II. RELATED WORK

A. The Computational Waste of Proof-of-Work

Proof-of-Work, the consensus mechanism pioneered by Bitcoin [3], secures decentralized networks by requiring miners to engage in a computationally intensive brute-force search for a valid block hash. The immense computational power expended in this process, which makes reversing transactions prohibitively expensive, is fundamental to its security model against threats like Sybil attacks [18]. However, this security comes at a great cost. The “work” itself—billions of arbitrary hash calculations—has no intrinsic value beyond network maintenance, leading to what is often described as large-scale computational and energy wastage [19].

The environmental and resource impact of this model is significant. For instance, a 2025 study highlighted that the carbon footprint of a single Bitcoin transaction can be equivalent to that of a mid-sized vehicle traveling over a thousand kilometers [20]. While alternatives such as Proof-of-Stake (PoS) [21],

[22] and Proof-of-Space (PoSpace) [23] have been proposed to mitigate energy use by replacing computational work with other scarce resources like economic collateral or storage, they fundamentally depart from the “work-based” security paradigm. In contrast, the PoUW approach, which this paper follows, seeks not to eliminate the work but to repurpose it, thereby directly addressing the core inefficiency of PoW by imbuing the computational effort with tangible, productive value.

B. Proof-of-Useful-Work

Conversely, an alternative interpretation of PoW’s energy consumption posits that this “wastage” is not merely a flaw but an intrinsic and indispensable feature of its security model. Building on this understanding, researchers have begun to explore new avenues that preserve this strong linkage to real-world costs while imbuing the cost-incurring process itself with additional productive value. Ball et al. formally introduced the concept of PoUW in 2017 [5]. The core tenet of PoUW is to replace the cryptographically hard puzzles of traditional PoW with computational tasks that possess practical application value. Early explorations in PoUW predominantly focused on mathematical problems and scientific computations. For instance, Primecoin [24] substituted hash computations with the search for specific prime number chains, while other work proposed leveraging blockchain computational power for tasks like genomic analysis [25]. While these endeavors successfully converted computational demands into tangible outputs, their limited economic models often failed to generate sufficient incentives to attract robust miner participation. To find more valuable and economically viable tasks, researchers increasingly turned to the computationally intensive field of Artificial Intelligence (AI). Proposals emerged to integrate machine learning model training into the consensus process, such as Proof of Learning (PoLe) [26]. However, these AI-based PoUW schemes introduced a critical new challenge: how to efficiently and verifiably confirm the correctness of complex computational work. Verifying model quality is often impractical due to the stochastic nature of training and high evaluation costs. Some solutions relied on heuristics or economic incentives [27], [28], but these often lack formal cryptographic guarantees and fall short of the robust, efficient verification found in traditional PoW. This tension between task “usefulness” and “verifiability” remains a central problem in PoUW design.

C. PoUW Based on Federated Learning

Distinct from centralized traditional machine learning, FL, as a distributed machine learning paradigm, has garnered significant research interest for its potential as a PoUW computational task. Qu et al. (2021) [14] introduced Proof of Federated Learning (PoFL), a game theory-based scheme that leverages the organizational similarities between FL and PoW mining pools to reclaim energy expended during consensus. In 2022, Wang et al. [15] further proposed Platform-free Proof of Federated Learning (PF-PoFL), enabling FL-based mining tasks to be performed in a fully decentralized manner. TORR, a lightweight blockchain framework for FL proposed by Ma

et al. in 2024 [16], features a lightweight consensus mechanism termed “Proof of Reliability,” based on erasure coding. Proof of Data (PoD), introduced by Liu et al. in 2025 [29], specifically addresses the Byzantine node threat present in prior FL-based PoUW schemes. While the aforementioned FL-based PoUW schemes ingeniously exploit the structural parallels between PoW blockchains and FL networks to mobilize and utilize blockchain computational resources, they predominantly rely on game theory and incentive mechanisms to ensure system stability and participant honesty. The core issue with this approach is that it substitutes the cryptographic guarantees of traditional PoW with economic assumptions, which can be less robust. The security of such systems hinges on the fragile assumption of participant rationality and can be vulnerable to complex collusion attacks, which are difficult to model exhaustively. Compared to original PoW, such designs often entail certain security compromises. In contrast, the scheme proposed in this study emphasizes the application of cryptographic techniques, aiming to achieve a security level comparable to traditional PoW mechanisms under the assumption of an attacker controlling no more than 50% of the total computational resources.

III. PRELIMINARIES

A. KZG Polynomial Commitment Scheme

We employ the Kate-Zaverucha-Goldberg (KZG) polynomial commitment scheme [17]. The KZG scheme enables a prover to commit to a polynomial $P(X) \in \mathbb{Z}_p[X]$ of degree less than d (for some integer d) by generating a succinct commitment C . Subsequently, the prover can convincingly demonstrate to a verifier the evaluation of $P(x)$ at an arbitrary point z , say $P(z) = y$, without revealing the polynomial $P(x)$ in its entirety.

B. BLS Aggregate Signature Scheme

This protocol utilizes the standard Boneh-Lynn-Shacham (BLS) scheme [30] for authenticity. It comprises four standard algorithms: **KeyGen**(λ) $\rightarrow$ (pk, sk); **Sign**(sk, M) $\rightarrow$ σ ; **Agg**($\{\sigma_i\}_{i=1}^n$) $\rightarrow$ σ_{agg} which efficiently compresses n signatures into a single aggregate signature; and **Verify**($\{(pk_i, M_i)\}_{i=1}^n, \sigma_{agg}$) $\rightarrow$ {True, False} which deterministically validates the aggregate signature against the respective public keys and messages.

IV. SYSTEM MODELS AND DESIGN GOALS

This section formally defines the system model of the PoUFW protocol, including participating entities, the communication model, security assumptions, and design goals. This model provides the necessary context for understanding the subsequent protocol description and the security properties.

A. System Model

As illustrated in Fig. 2, the PoUFW framework orchestrates three principal entities to facilitate federated learning while ensuring blockchain security:

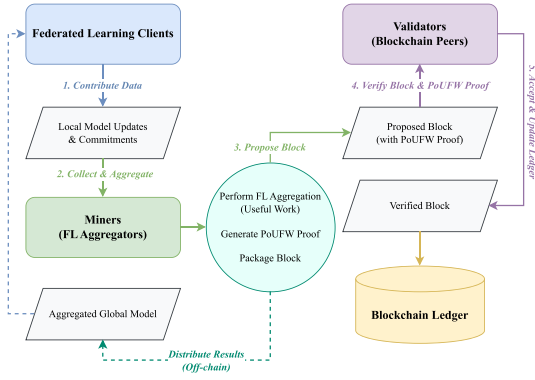


Fig. 2. PoUFW system model.

Clients are the originators of useful work. To contribute to the global model without exposing raw data, they perform local training for network-defined tasks. They encapsulate their updates using cryptographic commitments to ensure verifiability and privacy, authenticating them prior to submission to miners.

Miners act as federated learning aggregators and drive the PoUFW consensus. Incentivized by block rewards, they compete to collect authenticated local updates according to the network difficulty. Miners execute the core aggregation task and generate a concise, efficiently verifiable cryptographic proof of this aggregation. This proof and pending transactions are packaged into a candidate block, effectively replacing the energy-intensive hashing of traditional PoW with beneficial computational output.

Validators are distributed nodes that uphold blockchain consistency. Upon receiving a candidate block, they verify the PoUFW proof to confirm the aggregation's correctness without full re-execution, alongside standard transaction integrity checks. Validators can operate as *full nodes* for comprehensive validation or as *light nodes* to efficiently verify block headers and key proofs.

B. System-Level Federated Learning Workflow

To provide a holistic view of the PoUFW ecosystem beyond the core consensus mechanism, we outline the end-to-end lifecycle of a federated learning task within our blockchain framework:

- **Task Initiation & Initial Model:** A task publisher broadcasts an FL task request via a smart contract. This contract defines the neural network architecture, hyper-parameters, required difficulty targets (m_{req} , n_{req}), and the reward pool. The genesis block of the task anchors the initial global model parameters.
- **Client Discovery & Selection:** Clients discover active tasks by querying the P2P network. To prevent malicious miners from selectively excluding certain updates and to mitigate Sybil attacks while ensuring network liveness, the system employs a *Redundancy Pool* mechanism. Specifically, a larger candidate pool of eligible clients (e.g., $1.5 \times n_{req}$ clients) is deterministically selected for each epoch using a cryptographic pseudo-random function (PRF) seeded by

the previous block hash H_{prev} . Miners only need to collect valid updates from any n_{req} clients strictly within this redundancy pool to complete the block packing.

- **Model Distribution:** Selected clients retrieve the current global model from the decentralized storage layer (e.g., IPFS) using the Content Identifier (CID) stored in the latest block header, perform local training, and submit their cryptographic commitments.

C. Communication Model

We assume the PoUFW system operates on an open, peer-to-peer (P2P) network architecture, through which various participants in the system interconnect and exchange information. The communication process is modeled as asynchronous, meaning that message delivery between nodes may experience unpredictable network delays. Despite delays, we generally assume that messages sent by honest nodes will eventually reach their intended recipients, unless subjected to malicious blocking by network adversaries. Critical protocol messages, such as newly generated blocks, transactions, and client contribution metadata, rely on a standard peer-to-peer network broadcasting protocol – the Gossip protocol – for propagation throughout the network.

We acknowledge that network bandwidth is a limited resource, and the design of the PoUFW protocol (particularly placing large-scale aggregation result data processing off-chain, as detailed in Section V) has incorporated this constraint to promote system scalability. It is noteworthy that while consensus information and blocks are propagated through the main P2P network, the distribution of final aggregated results from miners to clients relies on supplementary off-chain communication mechanisms.

D. Threat Model

We consider a Probabilistic Polynomial-Time (PPT) adversary $\mathcal{A}$ seeking to compromise the system's correctness, security, or liveness. $\mathcal{A}$ controls less than half of the network's total computational power (denoted by a fraction $f_m < 1/2$) and corrupts fewer than half of the validation nodes (a fraction $f_v < 1/2$). The adversary has full access to public information, including protocol parameters, public keys, and the KZG Structured Reference String (SRS), but not to the private keys of honest entities or the SRS trapdoor.

$\mathcal{A}$ operates within an asynchronous network model, possessing standard capabilities such as eavesdropping, intercepting, delaying, replaying, or dropping messages. However, $\mathcal{A}$ cannot effect a complete network partition or indefinitely prevent communication among all honest nodes. Participants corrupted by $\mathcal{A}$ may arbitrarily deviate from protocol specifications. For example, malicious miners might attempt to forge aggregation results, generate invalid PoUFW proofs, propose blocks with invalid transactions, tamper with client contributions. Corrupted validators may attempt to affirm invalid blocks.

Regarding protocol execution, we assume clients are honest. However, in practical open federated learning deployments, malicious or Byzantine clients may attempt data poisoning

or model manipulation attacks [31]. It is essential to distinguish that PoUFW provides strong cryptographic guarantees for *computational integrity*, whereas ensuring *statistical robustness* at the data layer is an orthogonal challenge. To achieve robust FL, PoUFW can be integrated with preprocessing defense mechanisms like DAG-based virtual pruning as explored in ChainFL [32] or ledger-based reputation systems to filter out anomalous updates before verifiable aggregation. Furthermore, a malicious miner might attempt to selectively exclude specific client updates to bias the global model. As outlined in our system workflow, the protocol restricts client inclusion to a deterministically selected redundancy pool based on a pseudo-random seed derived from the previous block hash H_{prev} . Any block containing updates from clients outside this mandated candidate pool, or failing to collect exactly n_{req} valid updates from within it, will fail validation. This Redundancy Pool mechanism effectively bounds the miner's ability to arbitrarily cherry-pick favorable updates from the entire network, while simultaneously preserving network liveness even if some edge devices drop offline. All other honest participants are assumed to strictly adhere to the protocol.

E. Design Goals

The PoUFW protocol is designed to achieve the following key security and functional objectives:

- *Correctness*: Honest miners processing valid client inputs generate valid blocks that are subsequently accepted by honest validators.
- *Soundness*: A computationally bounded adversary cannot forge a valid PoUFW without performing the specified federated learning aggregation or an equivalently costly computation.
- *Adjustable Mining Difficulty*: Features a dynamic difficulty adjustment mechanism to maintain stable block generation times despite variations in network-wide computational power.
- *Usefulness of Work*: The computational effort expended during consensus, unlike traditional PoW, also generates valuable byproducts (e.g., trained machine learning models).
- *Verification Efficiency*: The verification of a PoUFW proof must be significantly faster and less resource-intensive than re-executing the complete aggregation computation.

V. DESIGN OF THE PoUFW PROTOCOL

To overcome the computational waste of traditional PoW without forcing validators into prohibitive FL re-executions, PoUFW leverages novel cryptographic techniques to enable robust, lightweight verification. This section details the protocol's execution mechanisms.

A. Setup

The KZG commitment scheme relies on a one-time trusted setup to generate the public parameters, known as the SRS. This process involves a secret random value $\tau \in \mathbb{Z}_p$, often termed

“toxic waste,” which must be discarded after the setup. The SRS comprises two components:

- $srs_1 = ([1]_1, [\tau]_1, [\tau^2]_1, \dots, [\tau^{d-1}]_1)$, a sequence of d elements from group G_1 .
- $srs_2 = ([1]_2, [\tau]_2)$, a pair of elements from group G_2 .

The parameter d must be set to exceed the maximum degree of any polynomial used in the protocol, such as those representing client updates ($P_i(z)$) and their aggregation ($P_{agg}(z)$).

Given the decentralized nature of the protocol, generating τ cannot be entrusted to a single party. Consequently, we adopt the standard approach of a Multi-Party Computation (MPC) ceremony [33]. This ceremony allows a group of mutually distrusting participants to collaboratively compute the SRS without any single party learning the underlying secret τ . In our design, this MPC is executed by the initial cohort of federated learning clients. The resulting SRS is then enshrined in the genesis block, establishing a non-malleable cryptographic foundation for all subsequent protocol operations.

B. PoUFW Consensus Round Execution

The specific execution workflow of consensus rounds underpins the PoUFW protocol. A characteristic PoUFW consensus round is designed to securely generate and validate new blockchain blocks through the competitive execution of useful federated learning aggregation tasks, fortified by efficient cryptographic verification. This section provides a detailed exposition of the constituent stages of a complete consensus round, specifying the roles, actions, and interactions of the involved participants, which culminates in achieving consensus on the new block's validity and advancing the ledger state. This process principally encompasses the following five interconnected stages: Client Contribution and Commitment, Mining (Federated Aggregation and Proof Generation), Block Proposal, Block Validation, and Chain Extension and Consensus Attainment as illustrated in Fig. 3.

B. Phase 1. Client Contribution & Commitment: In this phase, each client i participating in the current FL task (let $\mathcal{I}$ denote the set of participating clients) performs the following operations to prepare its contribution for the aggregation task and make it available for subsequent aggregation and validation:

Client i , in accordance with the current FL task requirements, performs computations based on its local dataset to obtain a local update $\mathbf{u}_i$ (which could be a gradient or model parameter update, determined by the specific task). Concurrently, the client determines its corresponding weight $w_i \in \mathbb{F}_p$ for the global aggregation, typically based on the size of its local dataset.

As polynomial commitments operate over the finite field $\mathbb{F}_p$, whereas the parameters of the local update $\mathbf{u}_i$ are typically represented as floating-point numbers, clients are required to scale them using fixed-point representation. A sufficiently large public scaling factor S is selected. Client i converts each element $u_{i,j}$ in $\mathbf{u}_i$ into an integer $x_{i,j} = \lfloor S \cdot u_{i,j} \rfloor$, where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer. This procedure maps $\mathbf{u}_i$ to an integer vector $\mathbf{x}_i = (x_{i,0}, x_{i,1}, \dots, x_{i,m-1}) \in \mathbb{Z}^m$. The elements of $\mathbf{x}_i$ can then be readily mapped to $\mathbb{F}_p$. For conciseness, $\mathbf{x}_i$ will henceforth denote the vector over $\mathbb{F}_p$. This quantization step

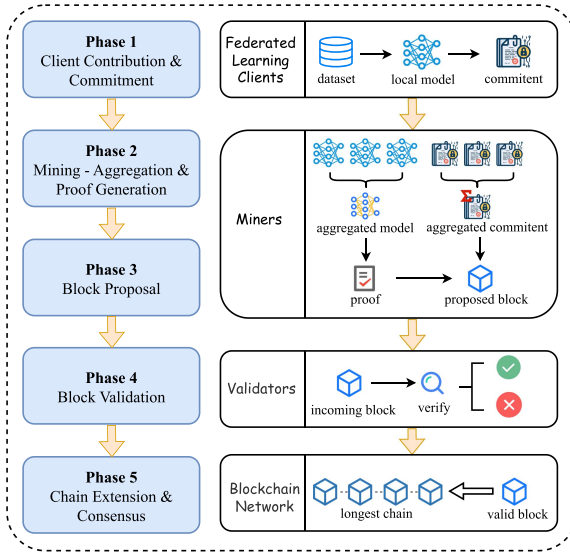


Fig. 3. Overview of a single consensus round in the PoUFW framework. The process integrates federated learning with blockchain consensus through five distinct phases. (1) **Client Contribution & Commitment**: Federated Learning clients train local models on their private data and submit them along with cryptographic commitments. (2) **Mining & Proof Generation**: Miners compete to perform the “useful work” of aggregating the collected models. The winning miner generates a proof of correct aggregation and packages it with the aggregated commitment into a new block proposal. (3) **Block Proposal**: The proposed block is broadcast to the network. (4) **Block Validation**: Validators verify the integrity of the incoming block, including the validity of the aggregation proof. (5) **Chain Extension & Consensus**: Upon successful validation, the block is appended to the longest chain, finalizing the FL model update and achieving network-wide consensus.

introduces a controllable precision loss but fundamentally enables subsequent cryptographic operations to maintain algebraic closure over the finite field $\mathbb{F}_p$. Conceptually, the choice of the scaling factor S dictates a trade-off between the representation precision of local updates and the available numerical range within the finite field. A substantially small S might excessively truncate fractional updates, potentially degrading the convergence rate or the final accuracy of the underlying FL model. Conversely, an adequately large S preserves sufficient gradient information, ensuring convergence behavior that is statistically indistinguishable from conventional floating-point aggregation. As we will empirically evaluate and validate in Section VII-E, selecting an appropriate S introduces minimal impact on model quality. Our work focuses on transforming federated learning aggregation tasks into useful work within a blockchain consensus system. While numerous methods exist to process local updates to prevent the server from inferring clients’ private information from these updates (e.g., differential privacy), we assume herein that the updates are protected by appropriate technical means.

Client i converts the fixed-point local update vector $\mathbf{x}_i = (x_{i,0}, x_{i,1}, \dots, x_{i,m-1})$ into a polynomial $P_i(z) \in \mathbb{F}_p[z]$, where the coefficients of the polynomial are the components of the vector:

$$P_i(z) = \sum_{j=0}^{m-1} x_{i,j} z^j. \quad (1)$$

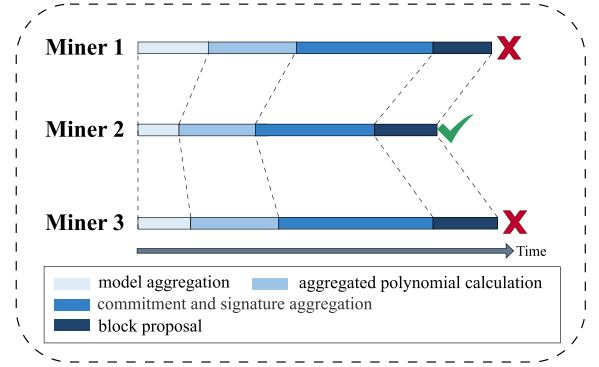


Fig. 4. The mining workflow in the PoUFW protocol, where Miner 2 is the first to complete the process, propose a new block, and secure the block authorship right.

Utilizing the public parameters (Structured Reference String, SRS) from the genesis block, client i computes a KZG commitment to its polynomial $P_i(z)$. This commitment, C_i , is an element of the elliptic curve group G_1 :

$$C_i = \text{Commit}(P_i(z)) = [P_i(\tau)]_1 = \sum_{j=0}^{m-1} x_{i,j} [\tau^j]_1. \quad (2)$$

We assume that each client i possesses a unique and long-term public-private key pair (pk_i, sk_i) , where the public key pk_i is bound to its identity and known to the network. To enable verification of the aggregation correctness, client i constructs a message $M_i = \text{Hash}(H_{prev} || w_i || C_i)$ where Hash is a collision-resistant hash function and computes a signature $\sigma_i = \text{Sign}(sk_i, M_i)$ using its secret key.

Clients do not proactively broadcast any data; instead, they await requests for the necessary data from miners who intend to compete for block authorship.

B. Phase 2. Mining - Aggregation and Proof Generation: In this phase, miner nodes perform the core useful work—federated learning aggregation—and generate cryptographic proofs to substantiate the validity of their work, thereby competing for block authorship as depicted in Fig. 4. The specific steps are as follows:

Miner $\mathcal{M}$ requests data from federated learning clients according to the current network difficulty. Assuming this difficulty requires the aggregation of local updates from n clients, $\mathcal{M}$ requests and obtains the dataset $\{(w_i, C_i, \sigma_i, \mathbf{x}_i)\}_{i=1}^n$ from n such clients. It is important to note that the total number of federated learning clients is at least n ; consequently, different miners may not necessarily acquire identical datasets.

Upon acquiring the dataset $\{(w_i, C_i, \sigma_i, \mathbf{x}_i)\}_{i=1}^n$ from n clients, miner $\mathcal{M}$ performs the aggregation. It computes the aggregated vector $\mathbf{x}_{agg} = \sum_{i=1}^n w_i \mathbf{x}_i \in \mathbb{F}_p^m$ and constructs the corresponding aggregated polynomial $P_{agg}(z) = \sum_{j=0}^{m-1} x_{agg,j} z^j \in \mathbb{F}_p[z]$. Utilizing the public SRS , $\mathcal{M}$ directly computes the aggregate KZG commitment $C_{agg} \in G_1$:

$$C_{agg} = \text{Commit}(P_{agg}(z)) = [P_{agg}(\tau)]_1 = \sum_{j=0}^{m-1} x_{agg,j} [\tau^j]_1. \quad (3)$$

In a fully zero-trust client scenario, miners would incur a computational overhead of $\mathcal{O}(n \cdot m)$ field operations to independently verify the well-formedness of individual client commitments. However, assuming clients format commitments correctly, miners may omit these checks to expedite proof generation. $\mathcal{M}$ then applies $\text{Agg}(\{\sigma_i\}_{i=1}^n)$ to produce the aggregated signature σ_{agg} .

Subsequently, miner $\mathcal{M}$ must prove that the computed $\mathbf{x}_{agg}$ accurately corresponds to the content represented by the commitment C_{agg} . This is accomplished by generating a KZG opening proof. The procedure utilizes our CPB mechanism, which employs the Fiat-Shamir heuristic to transform an interactive protocol into a non-interactive proof, thereby binding the proof to the specific miner and the current chain state.

To ensure the unpredictability and public verifiability of the challenge point c (derived below), and to bind it securely to the consensus context, miner $\mathcal{M}$ constructs a transcript incorporating the following elements:

- The miner's public key, pk_m : binding the proof to the identity of its generator.
- The hash of the preceding block, H_{prev} : forming the sequential linkage of the blockchain.
- The aggregated commitment, C_{agg} : pivotal for rendering the protocol non-interactive, as the challenge becomes dependent on the commitment itself.
- The Merkle tree root, $root_T$: this value is the root of the Merkle tree constructed over all transactions intended for inclusion in the proposed block's body. It anchors the proof to the specific block, thereby preventing replay attacks or reuse.

These elements are deterministically serialized to form the transcript $T = (pk_m, H_{prev}, C_{agg}, root_T)$. Miner $\mathcal{M}$ employs a hash function to generate the challenge point $c = H(T) \in \mathbb{F}_p$.

As all components of T are retrievable from the block to be proposed by the miner, any verifier can independently recompute the challenge point c .

$\mathcal{M}$ computes the evaluation of $P_{agg}(z)$ at the challenge point c , denoted $y \in \mathbb{F}_p$:

$$y = P_{agg}(c) = \sum_{j=0}^{m-1} x_{agg,j} c^j. \quad (4)$$

Then, the miner computes the KZG commitment $\pi = [Q(\tau)]_1 \in G_1$ to the quotient polynomial $Q(z) = \frac{P_{agg}(z) - y}{z - c}$, which serves as the opening proof. Miner $\mathcal{M}$ thereby obtains a non-interactive proof tuple (C_{agg}, y, π) .

B. Phase 3. Block Proposal: Upon successful completion of the aggregation computation and generation of a valid non-interactive proof of useful work, miner $\mathcal{M}$ proceeds to the block proposal phase. The objective of this phase is to translate the miner's computational results into a formal proposal for updating the blockchain state, thereby securing the right to append a new block.

Adhering to the network-defined block structure specifications, miner $\mathcal{M}$ initiates the construction of a new candidate

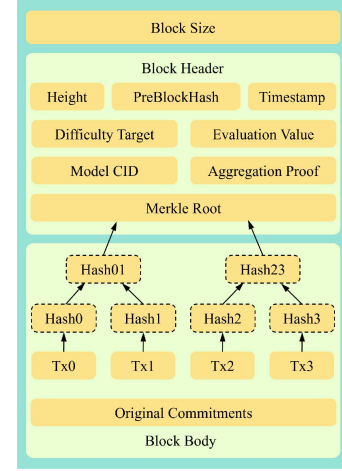


Fig. 5. The block structure of PoUFW.

block, B_{new} . This block incorporates standard blockchain meta-data and, crucially, encapsulates the essential information required to substantiate the performed useful work, the block structure is shown in Fig. 5.

The PoUFW block header includes conventional structural fields such as the previous block hash, timestamp, block height, and the Merkle tree root of transactions. Additionally, it incorporates novel fields that supersede elements typically associated with PoW in standard block headers:

- **Difficulty Target:** Replaces the difficulty target in a standard PoW block header. Consistent with the dynamic adjustment mechanism, this target exclusively regulates the mandated number of contributing clients, n_{req} , and the number of batched FL tasks.
- **Model CID:** The Content Identifier (CID) pointing to the aggregated global model parameters stored in the off-chain decentralized storage network. This fundamentally links the off-chain FL payload to the on-chain consensus state and enables clients to reliably retrieve the latest global model.
- **Aggregate Commitment:** The KZG commitment $C_{agg} \in G_1$ to the aggregated result $\mathbf{x}_{agg}$.
- **Evaluation Value:** The evaluation $y \in \mathbb{F}_p$ of the aggregated polynomial $P_{agg}(z)$ at the challenge point c .
- **Opening Proof:** The commitment $\pi \in G_1$ to the KZG quotient polynomial $Q(x)$, serving as the opening proof.

To support the Multi-Task Batching mechanism introduced in Section V-C, if a block encapsulates K independent FL tasks, the singular fields for Aggregate Commitment, Evaluation Value, and Opening Proof in the block header can be structurally replaced by a lightweight Merkle root of the K proof tuples, ensuring the header size remains strictly constant and scalable.

The PoUFW block body, aside from the customary transaction count and transaction list, incorporates an additional structure: **Original Client Commitments**. This structure archives the tuple pertaining to the commitments of the utilized clients, denoted as $(\{pk_i, w_i, C_i\}_{i=1}^n, \sigma_{agg})$.

Crucially, the block does not store any model parameters. This design choice obviates the substantial detrimental effects that

such voluminous data would impose on blockchain scalability and averts prohibitive communication bandwidth demands.

Prior to finalizing the block header, miner $\mathcal{M}$ stores the aggregated result x_{agg} on a decentralized storage network to obtain the corresponding Content Identifier (Model CID), which is then encapsulated into the candidate block B_{new} .

Once miner $\mathcal{M}$ finalizes the construction of B_{new} , it broadcasts the candidate block to its peers through its P2P network interface. The block subsequently disseminates across the network, typically via a gossip protocol, to reach a significant portion of the validating nodes. Clients can then retrieve x_{agg} directly using the CID enshrined in the block header, obviating substantial communication overhead from redundant off-chain transmissions.

Upon concluding this phase, miner $\mathcal{M}$ has encapsulated its performed useful work and the associated proof of validity within block B_{new} , which is then submitted to the distributed network for comprehensive verification and consensus determination. The architectural design of B_{new} ensures that all verifiers are equipped with the necessary information to independently and efficiently ascertain the validity of the PoUFW proof and the block's coherence with the existing on-chain state.

B. Phase 4. Block Validation: Upon receiving a candidate block B_{new} , validators first perform a series of standard consensus checks, including structural integrity, timestamp validity, previous block linkage, and transaction correctness. If these checks pass, they proceed to the core PoUFW-specific verification, which leverages data from the block's header and body.

All validators extract the PoUFW proof context from the block header: the aggregate commitment C_{agg} , the evaluation y , the opening proof π , and the components required for challenge reconstruction ($pk_m, H_{prev}, \text{root}_{\mathcal{T}}$). Full nodes additionally retrieve the original client commitment tuple ($\{pk_i, w_i, C_i\}_{i=1}^n, \sigma_{agg}$) from the block body to perform more comprehensive checks.

The PoUFW validation protocol comprises three sequential steps:

1) Aggregate Signature and Commitment Linearity (Full Nodes Only): Full nodes first verify the aggregate signature σ_{agg} against the client commitments and public keys. If valid, they then confirm the linearity of the aggregate commitment by checking if C_{agg} corresponds to the weighted sum of individual client commitments:

$$C_{agg} \stackrel{?}{=} \sum_{i=1}^n w_i C_i. \quad (5)$$

A failure in either check indicates a manipulated aggregation, leading to the block's rejection.

2) Challenge Reconstruction (All Nodes): All verifiers deterministically reconstruct the transcript $T = (pk_m, H_{prev}, C_{agg}, \text{root}_{\mathcal{T}})$ from the block context and recompute the challenge point $c = H(T)$. This ensures the challenge is bound to the block's unique content.

3) KZG Proof Verification (All Nodes): Finally, all validators utilize the public SRS and the reconstructed challenge c to verify the KZG proof. This is accomplished by checking the

fundamental pairing-based equation:

$$e(\pi, [\tau]_2 - c \cdot [1]_2) \stackrel{?}{=} e(C_{agg} - y \cdot [1]_1, [1]_2). \quad (6)$$

If this equation holds, the proof is valid, confirming that the miner correctly performed the aggregation work. A block is deemed valid and accepted only if it successfully passes all of these applicable verification steps.

3) Phase 5. Chain Extension and Consensus: Upon successful validation, a candidate block B_{new} is integrated into the network's consensus process. A node appends the valid block to its local version of the chain and updates its ledger state accordingly. To resolve inevitable network forks—a consequence of propagation delays and concurrent block discovery—the PoUFW protocol adheres to the canonical longest-chain rule. Nodes consistently recognize and extend the valid chain with the greatest cumulative proof of work as the single source of truth.

This mechanism ensures that consensus is a progressive and probabilistic process. With each subsequent block, or confirmation, the probability of a given block being removed from the canonical chain via a reorganization diminishes exponentially. After a sufficient number of confirmations, a block achieves *probabilistic finality*, rendering it a secure and effectively irreversible record suitable for high-assurance applications.

C. Difficulty Adjustment

Specifically, the baseline difficulty, D , of an FL aggregation task is defined by two primary workload determinants: the dimensionality of the model parameters, m , and the mandated number of contributing clients, n_{req} . Since m is intrinsically tied to the specific neural network architecture required by the application, dynamically adjusting it is fundamentally impractical. Consequently, difficulty regulation is achieved by dynamically adjusting the required client count n_{req} and leveraging multi-task batching for subsequent aggregation tasks.

The adjustment is executed at the end of each epoch, a fixed window of W blocks. All nodes compute the observed average block interval from the preceding epoch, T_{actual} , to derive an adjustment factor, $\mathcal{F}$:

$$\mathcal{F} = \frac{T_{target}}{T_{actual}}. \quad (7)$$

The target difficulty for the next epoch, D_{next} (where D scales primarily with n_{req}), is then computed based on the current difficulty, $D_{current}$, using the following formula:

$$D_{next} = D_{current} \cdot \max\left(\frac{1}{\alpha}, \min(\alpha, \mathcal{F})\right). \quad (8)$$

Here, $\alpha > 1$ is a damping factor that constrains the magnitude of any single adjustment, thereby mitigating volatile oscillations in network difficulty.

C. Difficulty Adjustment via Dynamic Client Scaling and Task Batching: In real-world deployments, dynamically adjusting the required model dimensionality m_{req} to match the fluid consensus difficulty is fundamentally impractical, as it would

force clients to frequently alter their local neural network architectures. Furthermore, introducing pseudo-random dummy padding to bridge the difficulty gap is cryptographically destructive: it mathematically pollutes the homomorphic linearity of the aggregated commitment and would inevitably force validators to execute massive scalar multiplications to verify the padding, thereby completely ruining the protocol's core lightweight verification property.

To elegantly decouple the consensus difficulty from the rigid constraints of specific FL models, PoUFW maintains a strictly fixed model dimension m dictated by the application layer. Instead, the protocol regulates mining difficulty strictly through useful dimensions: *Dynamic Client Scaling* and *Multi-Task Batching*.

First, the primary difficulty lever is the required number of contributing clients, n_{req} . By increasing n_{req} , the network proportionally scales the miner's useful computational workload (requiring $\mathcal{O}(n_{req} \cdot m)$ field operations for aggregation). Crucially, the validator's linearity check merely scales by $\mathcal{O}(n_{req})$ elliptic curve additions. Since $n_{req} \ll m$, the verification remains exceptionally lightweight and strictly independent of the massive model size m . As a significant corollary, a larger n_{req} mathematically enhances the statistical robustness and Byzantine fault tolerance of the global FL model.

Second, to absorb massive surges in network hash rate without inflating n_{req} beyond practical network bandwidth limits, PoUFW allows miners to flexibly batch multiple independent FL tasks (or model chunks) into a single block. The aggregate difficulty of a block is evaluated as $D = \sum_{k=1}^K m_k \cdot n_{req,k}$, where K is the number of batched tasks. This transforms the blockchain into a highly scalable, generalized FL computational platform, ensuring that 100% of the computational energy expended by miners is translated into verifiable, structurally pristine, and societally valuable machine learning outcomes without introducing any meaningless padding.

VI. THEORETICAL ANALYSIS

This section formally analyzes the security, correctness, and computational complexity of PoUFW. The analysis operates under the threat model defined in Section IV-D. We assume the computational hardness of Type 3 bilinear pairings (ensuring KZG's perfect binding and computational hiding) [17], the EUF-CMA security of the BLS signature scheme, and model the Fiat-Shamir heuristic within the Random Oracle Model (ROM). The adversary $\mathcal{A}$ is a Probabilistic Polynomial-Time (PPT) entity controlling strictly less than half of the network's computational power and validating nodes, while clients act honestly.

A. Soundness of PoUFW

Soundness guarantees that a computationally bounded adversary cannot forge a valid PoUFW block without genuinely executing the specified federated learning aggregation work.

Theorem 1 (Soundness of PoUFW): In the ROM, the probability that any PPT adversary $\mathcal{A}$ successfully generates a valid

PoUFW block without genuinely performing the requisite aggregation work is negligible in the security parameter λ .

Proof: Suppose $\mathcal{A}$ generates a forged block B' that passes all validation checks with a non-negligible probability $\epsilon(\lambda)$. This implies $\mathcal{A}$ successfully achieved at least one of the following forgeries: (a) Forged an individual signature σ_i to bypass the aggregate signature verification. (b) Evaded the linearity check $C_{agg} = \sum_{i=1}^n w_i C_i$ with an invalid C_{agg} . (c) Forged a KZG opening (y, π) for C_{agg} at challenge c satisfying the pairing equation despite $y \neq P_{agg}(c)$.

However, under the EUF-CMA security of the BLS scheme, the probability of (a), $\epsilon_a(\lambda)$, is negligible. Regarding (b), the linearity check $C_{agg} \stackrel{?}{=} \sum_{i=1}^n w_i C_i$ is a deterministic elliptic curve equality check; any mismatch will strictly fail the verification, rendering $\epsilon_b(\lambda)$ exactly 0. Furthermore, due to the computationally binding property of the KZG commitment under the q -Strong Diffie-Hellman (q -SDH) assumption, the valid commitment uniquely binds to the aggregated polynomial $P_{agg}(X)$ for any PPT adversary. Finally, the CPB mechanism leverages the Fiat-Shamir heuristic to derive the cryptographic challenge $c = H(pk_m, H_{prev}, C_{agg}, \text{root}_{\mathcal{T}})$. In the Random Oracle Model (ROM), H is modeled as a random oracle. The transcript inextricably links the challenge c to the specific block state. Any attempt to replay a valid proof for a different block or by a different miner alters the context, yielding an unpredictable challenge c' . To forge a valid evaluation proof (y, π) for c' such that $y \neq P_{agg}(c')$, $\mathcal{A}$ must either find a collision in the hash function or break the q -Strong Diffie-Hellman (q -SDH) assumption of the KZG scheme, making the probability of (c), $\epsilon_c(\lambda)$, also negligible. Thus, $\mathcal{A}$'s total success probability $\epsilon(\lambda) \leq \epsilon_a(\lambda) + \epsilon_b(\lambda) + \epsilon_c(\lambda)$ is negligible. ■

B. Correctness of PoUFW

Correctness ensures that PoUFW blocks generated by honest miners will invariably pass all verification checks by honest verifiers.

Theorem 2 (Correctness of PoUFW): Any candidate block B_{new} generated by an honest miner strictly adhering to the PoUFW protocol with valid client inputs will pass all verification checks performed by honest verifiers.

Proof: An honest miner correctly computes the aggregated polynomial $P_{agg}(z) = \sum_{i=1}^n w_i P_i(z)$ and aggregates valid signatures. By the linearity of KZG commitments, the aggregated commitment naturally satisfies $C_{agg} = [P_{agg}(\tau)]_1 = \sum_{i=1}^n w_i [P_i(\tau)]_1 = \sum_{i=1}^n w_i C_i$, inherently passing the full node's linearity check and signature verification.

For the KZG proof, the deterministic challenge point $c = H(T)$ is identically reconstructed by verifiers. The honest miner correctly evaluates $y = P_{agg}(c)$ and computes the quotient polynomial $Q(z) = \frac{P_{agg}(z) - y}{z - c}$. Evaluated at the secret τ , this yields the identity $P_{agg}(\tau) - y = (\tau - c)Q(\tau)$. Applying the properties of bilinear pairings, the verifier's check simplifies to:

$$\begin{aligned} e(\pi, [\tau]_2 - c \cdot [1]_2) &= e([Q(\tau)]_1, [\tau - c]_2) \\ &= e([1]_1, [1]_2)^{Q(\tau)(\tau - c)}, \end{aligned}$$

$$\begin{aligned} e(C_{agg} - y \cdot [1]_1, [1]_2) &= e([P_{agg}(\tau) - y]_1, [1]_2) \\ &= e([1]_1, [1]_2)^{P_{agg}(\tau) - y}. \end{aligned}$$

Since $Q(\tau)(\tau - c) = P_{agg}(\tau) - y$, the verification equation strictly holds. Combined with standard block checks, B_{new} is uniformly accepted. ■

C. Computational Complexity Analysis

We analyze the computational overhead based on model dimensionality m and the number of clients n . Let E_1 denote a scalar multiplication in G_1 , P a bilinear pairing (where $P \gg E_1$), and V an aggregate signature verification.

- **Client:** The workload is dominated by computing the KZG commitment for the local model update vector, requiring m scalar multiplications. The complexity is $\mathcal{O}(mE_1)$.
- **Miner:** The miner aggregates n parameter vectors ($\mathcal{O}(nm)$ field operations), constructs the aggregate commitment ($\mathcal{O}(nE_1)$), and generates the opening proof ($\mathcal{O}(mE_1)$). The total computational complexity is intensively scaled at $\mathcal{O}(nm + (n + m)E_1)$.
- **Verifier:** PoUFW achieves highly asymmetric verification. A *Full Node* verifies the aggregate signature ($\mathcal{O}(V)$), checks commitment linearity ($\mathcal{O}(nE_1)$), and validates the KZG proof with a constant number of pairings ($\mathcal{O}(P)$), yielding $\mathcal{O}(V + nE_1 + P)$. A *Light Node* only validates the block header via the KZG proof check, requiring a constant-time complexity of $\mathcal{O}(P)$, completely independent of m and n .

This analysis confirms that while the miner's proof generation is computationally intensive (serving as robust PoW), the verification remains exceptionally lightweight. This asymmetry is fundamental to the protocol's high scalability.

VII. EXPERIMENTAL EVALUATION

To experimentally validate the practical performance of the proposed PoUFW protocol, this section presents a comprehensive performance evaluation. Through a series of simulation experiments, we meticulously examine the protocol's performance across several critical dimensions. These include the computational efficiency of clients, miners, and verifiers, with a particular emphasis on the lightweight characteristics of the verification process. Concurrently, we analyze the protocol's scalability under diverse task configurations, thereby furnishing empirical data to support the viability of PoUFW for real-world applications.

Simulations were conducted on an Ubuntu 24.04 workstation equipped with an AMD Ryzen Threadripper 3960X CPU, 32 GB RAM, and an NVIDIA RTX 3090 GPU. The PoUFW protocol was implemented in Rust (*rustc 1.87.0-nightly*), utilizing the *bls12-381* pairing-friendly elliptic curve for all cryptographic operations. Based on our complexity analysis in Section VI-C, we primarily investigate the impact of model dimensionality m and the number of clients n on protocol performance.

A. Impact of Parameter Size on Performance

Initially, to assess the influence of parameter size (m), the number of federated learning clients (n) was held constant at representative values (e.g., 100, 200, and 300). These represent typical deployments in medium-to-large-scale federated learning contexts. The experimental outcomes, illustrated in Fig. 6, distinctly delineate the correlation between the parameter size and the operational latency experienced by each category of participant. Specifically:

a) **Federated Learning Client & Miner:** As illustrated in Fig. 6, operational latencies for clients and miners scale linearly with parameter size m . For instance, at $m = 460,000$, client processing requires ~ 374 ms dominated by $\mathcal{O}(m)$ commitment generation, while miner proof generation takes ~ 14.8 seconds. This substantial miner duration constitutes the robust "useful work" underlying consensus difficulty. Varying the client count n induces negligible latency fluctuation for a fixed m .

b) **Verifier:** Crucially, verifier latency demonstrates strict independence from m , consistently requiring merely 6.0–14.7 ms depending on n . This aligns precisely with our theoretical $\mathcal{O}(1)$ verification complexity regarding m (Section VI-C), corroborating PoUFW's highly efficient validation.

B. Impact of Client Count on Performance

Following this, to evaluate the influence of the number of federated learning clients (n) participating in a single consensus round on the protocol's performance, we fixed the parameter size (m) at representative values. The experimental results are presented in Fig. 7. Specifically:

a) **Impact on Provers (Clients and Miners):** As depicted in Fig. 7, fixing m reveals that individual client latency is entirely invariant to the total network size n . Concurrently, miner processing time exhibits only a marginal increase as n scales (e.g., escalating by ~ 272 ms as n grows from 10 to 910 at $m = 300,000$). This confirms that m , rather than n , strictly dominates the consensus workload.

b) **Impact on Verifiers (Full Nodes):** For full nodes, the aggregate commitment linearity check ($\mathcal{O}(nE_1)$) induces latency directly proportional to n . Nonetheless, validation remains exceptionally lightweight, completing in ~ 25 ms even at an extreme scale of $n = 910$, ensuring rapid block propagation.

C. Analysis of Consensus Latency and Component Contributions

Fig. 8 provides an intuitive depiction of how single-round consensus latency varies with changes in both parameter size (m) and the number of clients (n). Furthermore, it disaggregates the total latency, revealing the proportional contribution of operations performed by different participants. The depicted trends in Fig. 8, based on representative configurations, affirm, in line with our preceding theoretical discussions in Section VI-C, that the parameter size (m) exerts a considerably more pronounced influence on overall consensus latency compared to the number of clients (n). Consequently, although the parameter size (m) exerts a more pronounced influence on computational latency,

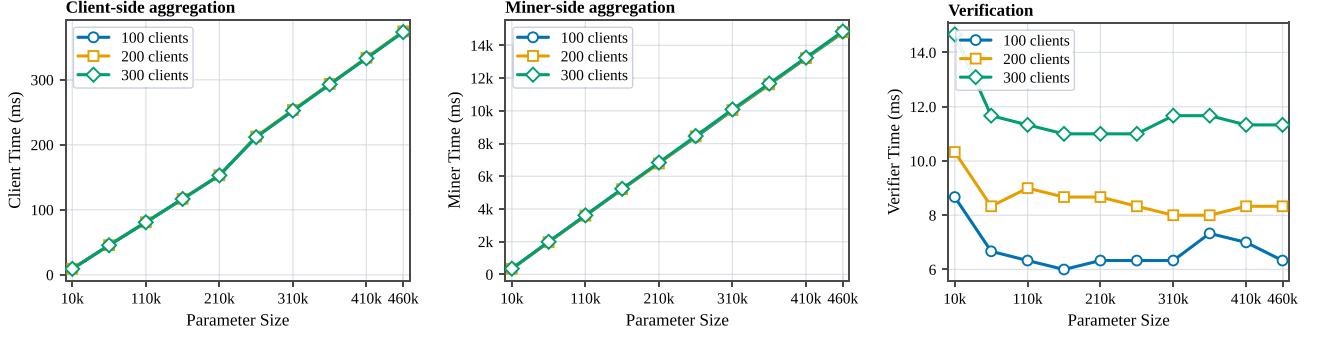


Fig. 6. Operational time of consensus participants versus parameter size (m) for varying numbers of clients (n) in a simulated FedAvg task. For context in real-world FL tasks, $m \approx 100,000$ represents a lightweight IoT model, while $m \approx 460,000$ corresponds to more complex architectures.

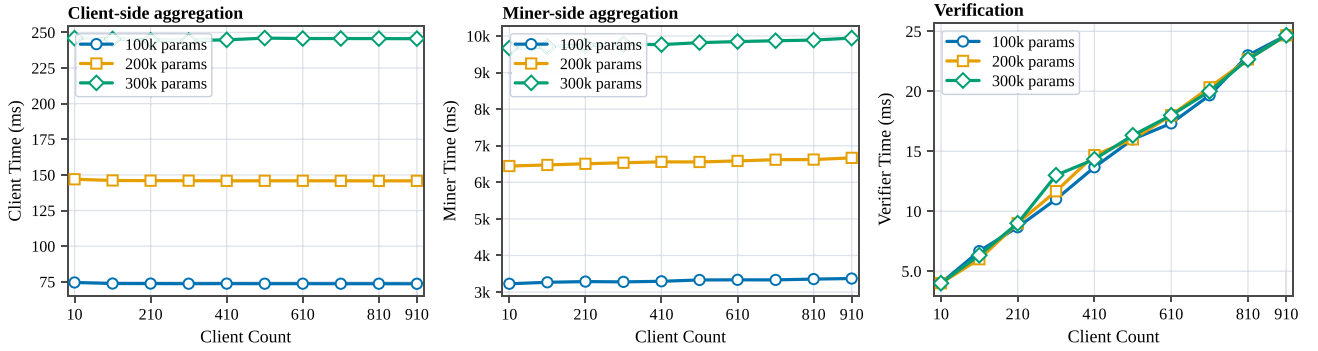


Fig. 7. Operational time of consensus participants versus the number of clients (n , reflecting typical cross-device FL participation scales) for varying fixed parameter sizes (m).

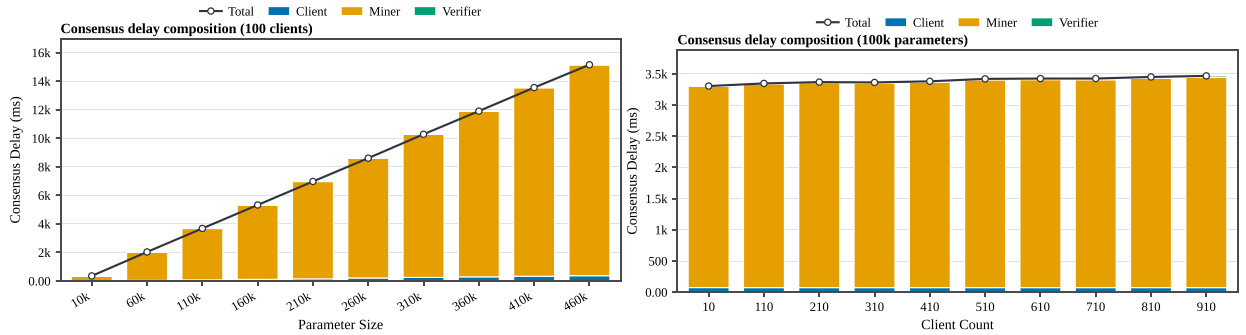


Fig. 8. Single-round consensus latency variation with parameter size m and the number of clients n . The figure illustrates the stronger influence of m compared to n and highlights the dominant contribution of the miner's operational time to the total latency, consistent with the PoUW design philosophy.

dynamically adjusting it is impractical as it is strictly fixed by the specific FL application. Instead, as discussed in Section V-C, significant recalibration of the mining difficulty is effectively achieved through the Multi-Task Batching mechanism, whereas modifications to the client count (n_{req}) provide finer-grained control.

D. Evaluation of Storage and Communication Overhead

To comprehensively address the practical scalability of the PoUFW protocol, we evaluated the storage overhead of the cryptographic parameters and the impact of communication on consensus latency as the model dimensionality (m) scales.

1) Cryptographic Storage Asymmetry: The KZG polynomial commitment scheme requires a one-time trusted setup to generate the Structured Reference String (SRS). As illustrated in Fig. 9, PoUFW exhibits a profound cryptographic storage asymmetry. For the prover (miner), the SRS size grows linearly ($\mathcal{O}(m)$) with the model dimensionality. At $m = 460,000$, the prover's SRS requires approximately 21.06 MB, scaling to roughly 45.7 MB for $m = 1,000,000$. While this necessitates miners to store the SRS locally, this storage footprint is highly manageable for financially incentivized nodes equipped with standard hardware. Furthermore, the SRS distribution is a one-time operation during the initial setup phase, meaning it does not consume recurring network bandwidth during consensus rounds.

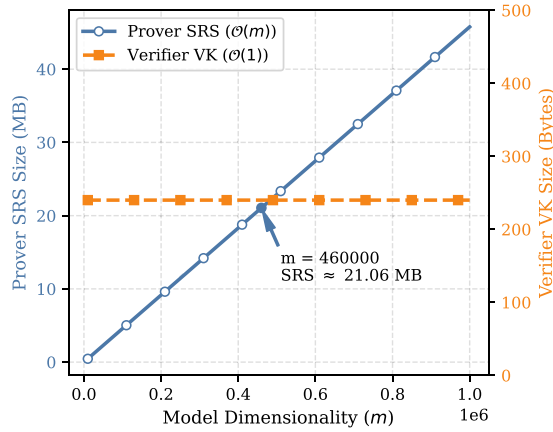


Fig. 9. Cryptographic storage asymmetry in PoUFW. The Prover's SRS size scales linearly with the model dimensionality (m), while the Verifier Key size remains strictly constant at approximately 240 bytes, ensuring lightweight validation.

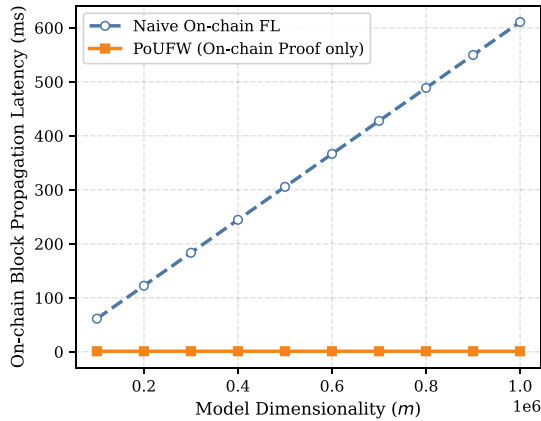


Fig. 10. Impact of communication overhead on consensus latency (simulated at 50 Mbps). PoUFW maintains near-zero on-chain block propagation latency by storing only constant-size proofs, avoiding the linear delay penalty of naive on-chain FL schemes.

Crucially, the Verifier Key (VK) size remains strictly constant ($\mathcal{O}(1)$) at approximately 240 bytes, completely independent of the model size. This strict asymmetry guarantees that validators and resource-constrained light nodes incur negligible storage and memory overhead, fully preserving the protocol's high decentralization and lightweight verification properties.

2) *Impact of Communication on Consensus Latency:* In blockchain networks, embedding massive amounts of data into blocks significantly increases propagation delay, which degrades consensus efficiency and elevates fork rates. To evaluate this communication impact, we simulated on-chain block propagation latency over a standard 50 Mbps network connection.

As depicted in Fig. 10, a naive on-chain FL approach, which directly embeds the aggregated model parameters into the block body, suffers from severe propagation delays that scale linearly with the model size, exceeding 600 ms for $m = 1,000,000$. In stark contrast, PoUFW securely decouples the voluminous FL

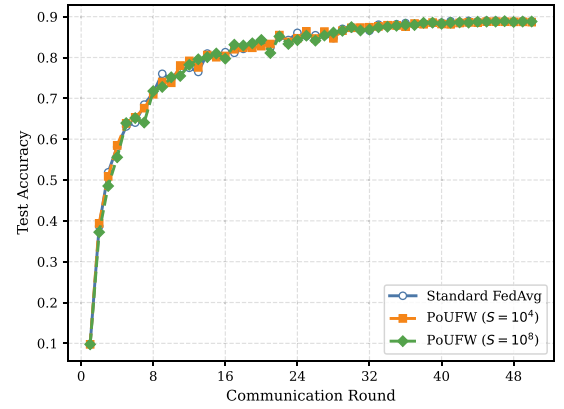


Fig. 11. Test accuracy over communication rounds for ResNet-20 on the CIFAR-10 dataset. The performance of PoUFW aggregation with scaling factors $S = 10^4$ and $S = 10^8$ is compared against the standard unquantized FedAvg baseline, demonstrating that PoUFW tightly preserves model quality and convergence behavior.

model data from the consensus layer. By storing only constant-size cryptographic proofs on-chain, PoUFW's on-chain communication footprint remains minimal ($\mathcal{O}(1)$). Consequently, the block propagation latency stays near zero, effectively insulating the consensus mechanism from the massive communication bottlenecks typically associated with large-scale federated learning tasks. The actual aggregated parameters are distributed asynchronously via off-chain distributed storage.

E. Evaluation of Model Quality and Convergence

A fundamental premise of the Proof-of-Useful-Work paradigm is that the consensus mechanism must not inadvertently degrade the utility of the underlying task. To empirically validate the practical utility of the PoUFW aggregation and address the impact of the fixed-point scaling factor S introduced in Section V-B on model precision, we conducted an image classification experiment. The task was performed on the CIFAR-10 dataset using a ResNet-20 neural network architecture.

As detailed in Phase 1, PoUFW requires mapping floating-point local updates to integers in a finite field $\mathbb{F}_p$ via a scaling factor S . We compared the standard unquantized FedAvg algorithm using native 32-bit floating-point precision against the PoUFW aggregation mechanism utilizing two distinct scaling factors: $S = 10^4$ and $S = 10^8$.

As illustrated in Fig. 11, the test accuracy curves of the PoUFW protocol across communication rounds tightly track the baseline Standard FedAvg. Both the $S = 10^4$ and $S = 10^8$ configurations achieve convergence rates and final model test accuracies (approximately 89%) that are virtually indistinguishable from the unquantized baseline. This empirically demonstrates that the fixed-point quantization requisite for the KZG polynomial commitments in PoUFW introduces statistically negligible precision loss. Consequently, the protocol successfully binds computational effort to robust consensus security while guaranteeing that the generated machine learning models retain high practical value.

VIII. CONCLUSION

This paper introduces a novel consensus protocol, termed Proof-of-Useful-Federated-Work (PoUFW), designed to address the inherent issue of computational resource wastage prevalent in traditional PoW mechanisms. PoUFW innovatively repurposes aggregation tasks from FL to serve as the foundational work in its proof mechanism. By ingeniously employing the KZG polynomial commitment scheme, it facilitates succinct, efficient, and non-interactive cryptographic verification of this aggregation work. This design paradigm not only converts the substantial computational expenditure typically incurred in securing consensus into tangible, valuable outcomes in distributed machine learning but also pioneers novel approaches for enhancing the decentralization of and incentive structures within federated learning.

Rigorous theoretical analysis has substantiated the protocol's security and correctness. Furthermore, extensive simulation experiments have systematically evaluated the computational overhead and performance metrics of the PoUFW protocol across diverse parameter configurations. The findings demonstrate that PoUFW not only upholds security but also achieves remarkable verification efficiency, indicating its feasibility and high performance for real-world deployment. Consequently, this work furnishes robust theoretical and empirical foundations for the development of blockchain systems characterized by enhanced societal value and superior resource utilization.

REFERENCES

- [1] A. Bathula et al., "Blockchain, artificial intelligence, and healthcare: The tripod of future—A narrative review," *Artif. Intell. Rev.*, vol. 57, no. 9, Aug. 2024, Art. no. 238.
- [2] M. Xu et al., "Exploring blockchain technology through a modular lens: A survey," *ACM Comput. Surv.*, vol. 56, no. 9, pp. 1–39, 2024.
- [3] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. Accessed: Apr. 01, 2026. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [4] "Cambridge blockchain network sustainability index: CBECI," (n.d.). [Online]. Available: <https://ccaf.io/cbnsi/cbeci>
- [5] M. Ball, A. Rosen, M. Sabin, and P. N. Vasudevan, "Proofs of useful work," (n.d.). [Online]. Available: <https://eprint.iacr.org/2017/203>
- [6] F. Hoffmann, "Challenges of proof-of-useful-work (PoUW)," in *Proc. IEEE Glob. Emerg. Technol. Blockchain Forum: Blockchain Beyond*, Nov. 2022, pp. 1–5.
- [7] Y. Zhao, Y. Qu, Y. Xiang, F. Chen, and L. Gao, "Context-aware consensus algorithm for blockchain-empowered federated learning," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 491–503, Apr.–Jun. 2024.
- [8] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, Apr. 2017, pp. 1273–1282.
- [9] M. M. Orabi, O. Emam, and H. Fahmy, "Adapting security and decentralized knowledge enhancement in federated learning using blockchain technology: Literature review," *J. Big Data*, vol. 12, no. 1, 2025, Art. no. 55.
- [10] E. T. Martínez Beltrán et al., "Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2983–3013, Fourthquarter 2023.
- [11] J. Li et al., "Blockchain assisted decentralized federated learning (BLADE-FL): Performance analysis and resource allocation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2401–2415, Oct. 2022.
- [12] Z. Wang, Q. Hu, R. Li, M. Xu, and Z. Xiong, "Incentive mechanism design for joint resource allocation in blockchain-based federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 5, pp. 1536–1547, May 2023.
- [13] S. Kayikci and T. M. Khoshgoftar, "Blockchain meets machine learning: A survey," *J. Big Data*, vol. 11, no. 1, 2024, Art. no. 9.

- [14] X. Qu, S. Wang, Q. Hu, and X. Cheng, "Proof of federated learning: A novel energy-recycling consensus algorithm," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 8, pp. 2074–2085, Aug. 2021.
- [15] Y. Wang, H. Peng, Z. Su, T. H. Luan, A. Benslimane, and Y. Wu, "A platform-free proof of federated learning consensus mechanism for sustainable blockchains," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3305–3324, Dec. 2022.
- [16] X. Ma and D. Xu, "TORR: A lightweight blockchain for decentralized federated learning," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 1028–1040, Jan. 2024.
- [17] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, M. Abe, Ed., 2010, pp. 177–194.
- [18] A. Back, "HashCash - A denial of service counter-measure," 2002. Accessed: Apr. 01, 2026. [Online]. Available: <http://www.hashcash.org/papers/hashcash.pdf>
- [19] G. A. F. Rebello et al., "A survey on blockchain scalability: From hardware to layer-two protocols," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2411–2458, Fourthquarter 2024.
- [20] N. C. Onat, R. Jabbar, M. Kucukvar, T. Wakjira, A. A. Kutty, and N. Fetais, "Carbon footprint of global bitcoin mining: Emissions beyond borders," *Sustainability Sci.*, vol. 20, no. 1, pp. 173–189, Jan. 2025.
- [21] S. King and S. Nadal, "PPCoin: Peer-to-peer crypto-currency with proof-of-stake," self-published white paper, Aug. 19, 2012, pp. 1–6. [Online]. Available: <https://www.peercoin.net/papers/peercoin-paper.pdf>
- [22] C. Schwarz-Schilling, J. Neu, B. Monnot, A. Asgaonkar, E. N. Tas, and D. Tse, "Three attacks on proof-of-stake ethereum," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2022, pp. 560–576.
- [23] S. Dziembowski, S. Faust, V. Kolmogorov, and K. Pietrzak, "Proofs of space," in *Proc. Annu. Cryptol. Conf.*, R. Gennaro and M. Robshaw, Eds. New York, NY, USA: Springer, 2015, pp. 585–605.
- [24] S. King, "Primecoin: Cryptocurrency with prime number proof-of-work," white paper, Jul. 7, 2013, pp. 1–6. [Online]. Available: <https://primecoin.io/primecoin-paper.pdf>
- [25] H. I. Ozercan, A. M. Ileri, E. Ayday, and C. Alkan, "Realizing the potential of blockchain technologies in genomics," *Genome Res.*, vol. 28, no. 9, pp. 1255–1263, Jan. 2018.
- [26] Y. Lan, Y. Liu, B. Li, and C. Miao, "Proof of learning (PoLe): Empowering machine learning with consensus building on blockchains (Demo)," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 18, pp. 16063–16066, May 2021.
- [27] Y. Wu, S. Choi, G. Liu, and X. Wang, "Proof of directed guiding gradients: A new proof of learning consensus mechanism with constant-time verification," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency*, May 2023, pp. 1–3.
- [28] Z. Zhao et al., "Proof-of-learning with incentive security," May 4, 2026, *arXiv:2404.09005*.
- [29] H. Liu, F. Zhu, and L. Cheng, "Proof-of-data: A consensus protocol for collaborative intelligence," Jan. 6, 2025, *arXiv:2501.02971v1*.
- [30] D. Boneh, C. Gentry, B. Lynn, and H. Shacham, "Aggregate and verifiably encrypted signatures from bilinear maps," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Warsaw, Poland, 2003, pp. 416–432.
- [31] M. P. Uddin, Y. Xiang, M. Hasan, J. Bai, Y. Zhao, and L. Gao, "A systematic literature review of robust federated learning: Issues, solutions, and future research directions," *ACM Comput. Surv.*, vol. 57, no. 10, pp. 1–62, 2025.
- [32] S. Yuan, B. Cao, Y. Sun, Z. Wan, and M. Peng, "Secure and efficient federated learning through layering and sharding blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 3120–3134, May/Jun. 2024.
- [33] S. Bowe, A. Gabizon, and I. Miers, "Scalable multi-party computation for zk-SNARK parameters in the random beacon model," *Cryptology ePrint Archive*, Paper 2017/1050, 2017. [Online]. Available: <https://eprint.iacr.org/2017/1050>



Hengrui Lu received the BS degree in cyberspace security in 2021 from Xidian University, Xi'an, China, where he is currently working toward the PhD degree with the School of Cyberspace Security. His research interests include the areas of blockchain, distributed systems, and applied cryptography.



Kai Fan (Member, IEEE) received the BS degree in telecommunication engineering, the MS degree in cryptography, and PhD degree in telecommunication and information system from Xidian University, China, in 2002, 2005, and 2007, respectively. He is currently a professor with the State Key Laboratory of Integrated Service Networks, Xidian University. He authored or coauthored more than 70 papers in journals and conferences, and holds nine Chinese patents. He has also managed five National Research Projects. His research interests include IoT security and information security.



Kuan Zhang (Member, IEEE) received the PhD degree in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada, in 2016. From 2017 to 2023, he was an assistant professor with the University of Nebraska-Lincoln, USA, where he is currently an associate professor with the Department of Electrical and Computer Engineering. He has authored or coauthored more than 100 papers in journals and conferences. His research interests include cyber security, AI, and cloud/edge computing. Dr. Zhang was the recipient of the Best Paper Award at IEEE WCNC 2013, Securecomm 2016, and ICC 2020. He was the Editor-in-Chief of *Encyclopedia of Wireless Networks*. He is also an associate editor for *IEEE Transactions on Wireless Communications*, *IEEE Internet of Things*, *IEEE Communications Survey and Tutorials*, and *Peer-to-Peer Networking and Application*. He was also a symposium co-chair of IEEE ICC and CIC/ICCC.



Zhanpeng Guo received the BS degree in mechanical engineering from the Nanjing University of Aeronautics and Astronautics, Jiangsu Province, China, the master's degree in network and information security from Xidian University, Shaanxi Province, China. His research focuses on applied cryptography, including zero-knowledge proofs and digital signatures.



Hui Li (Member, IEEE) received the BSc degree from Fudan University, in 1990, and the MSc and PhD degrees from Xidian University, China, in 1993 and 1998, respectively. Since 2005, he has been a professor with the School of Telecommunication Engineering, Xidian University. His research interests include cryptography, wireless network security, information theory, and network coding. He was the TPC Co-Chair for ISPEC 2009 and IAS 2009, general co-chair for E-Forensic 2010, ProvSec 2011, and ISC 2011, and honorary chair for NSS 2014 and ASIACCS 2016.